

For Loops

Meeting 3

Attribution: Developed by Julin Maloof, modified by Daniel Koenig

For Loops

Often it is necessary to repeat a computational task many times with subtle variation or to perform the same task on a large number of objects or files.

For example, you might need to:

- Run **BLAST** many times with different word sizes
- **BLAST** a gene against many different genomes, each separately
- Map 100 Illumina sequence files against a reference genome file
- Make a separate plot of DNA methylation for each of the 26 human chromosomes
- There are many other examples that you will encounter...

For Loops

Imagine that we are actually together in a classroom and we want to use a computer and robot to automatically measure the weight of every student in the room.

First lets describe in detail what the steps would be for a single student. We use “pseudo-code” to describe what we want our program to do for a single student.

For Loops

Imagine that we are actually together in a classroom and we want to use a computer and robot to automatically measure the weight of every student in the room.

First lets describe in detail what the steps would be for a single student. We use “pseudo-code” to describe what we want our program to do for a single student.

```
pick up the student  
bring them to the scale  
record their weight  
return them to their seat
```

For Loops

How would we describe the process of measuring all the students?

```
for each student in the classroom:  
    pick up the student  
    bring them to the scale  
    record their weight  
    return them to their seat
```

Note the use of the word **for**. This is natural English in this context, but it is also why these are called **for loops** in computer languages.

In this example student is a variable that takes on a different value each time we go through the loop.

For Loop example

Lets try it! Unfortunately I don't have a robot or scale, so we will just have the computer pretend that it is doing the task and tell us what it is doing.

Type the commands below into the Linux shell to see what happens

```
classroom_students="Mary Lisa John Colton Eli Mishi Tyler"  
echo ${classroom_students}
```

First we create a list of all the students, contained in the variable `classroom_students`

Remember that a *variable* is an object that contains some content (that can vary...)

After defining a variable we refer to it by placing a `$` in front of it and (optionally) enclosing it with `{ }`. This tells the Linux bash shell to fetch the contents of the variable.

We use the `echo` command to confirm that the `classroom_students` list has been created successfully

For Loop example

```
for student in ${classroom_students}
do
    echo "picking up ${student}"
    echo "bringing ${student} to the scale"
    echo "recording ${student}'s weight"
    echo "returning ${student} to their seat"
    echo
done
```

For Loop example

```
for student in ${classroom_students}
do
    echo "picking up ${student}"
    echo "bringing ${student} to the
scale"
    echo "recording ${student}'s weight"
    echo "returning ${student} to their
seat"
    echo # adds a blank line between
students
done
```

The variable `classroom_students` contains six students

The for loop takes the first student from `classroom_students` (Mary) and sets the variable `student` to match

The code between `do` and `done` is then run with `student` set to “Mary”.

Everywhere there is a `${student}` in the code, bash will substitute “Mary”.

After running the code with “Mary”, bash takes us back to the `do` statement, gets the next student (Lisa)

The code between `do` and `done` is now run again but with “Lisa” as the student.

The process repeats until every student in `classroom_students` has been processed.

Work for today

- 1) If you haven't completed git or Linux tutorial do those first
- 2) Do the For Loop tutorial
- 3) BLAST tutorial